

Programmazione 1

Corso c

02/10/2023

> Dai numeri ai Puntatori

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Numeri: float, aritmetica in virgola mobile, math.h
- Conversioni di tipo
- L'operatore di indirizzo
- Input da tastiera
- I puntatori

Outline

- Numeri: float, aritmetica in virgola mobile, math.h
- Conversioni di tipo
- L'operatore di indirizzo
- Input da tastiera
- I puntatori

I tipi float e double

- Nel calcolo scientifico è fondamentale poter lavorare con i numeri reali, ovvero con la virgola (*floating point*)
- I tipi *float* e *double* in C permettono di rappresentare numeri reali con precisione a 32 e 64 bit rispettivamente
- Lo standard IEEE 754 definisce la rappresentazione dei numeri e la procedura di conversione decimale <-> binario
- Valgono gli operatori aritmetici $+$ $-$ $*$ $/$ $\%$ visti per *int*
- Stampabili via *printf* con gli specificatori di formato **%f** (*fixed-point notation*), **%e** (*exponential notation*) e **%g** (*mixed*)

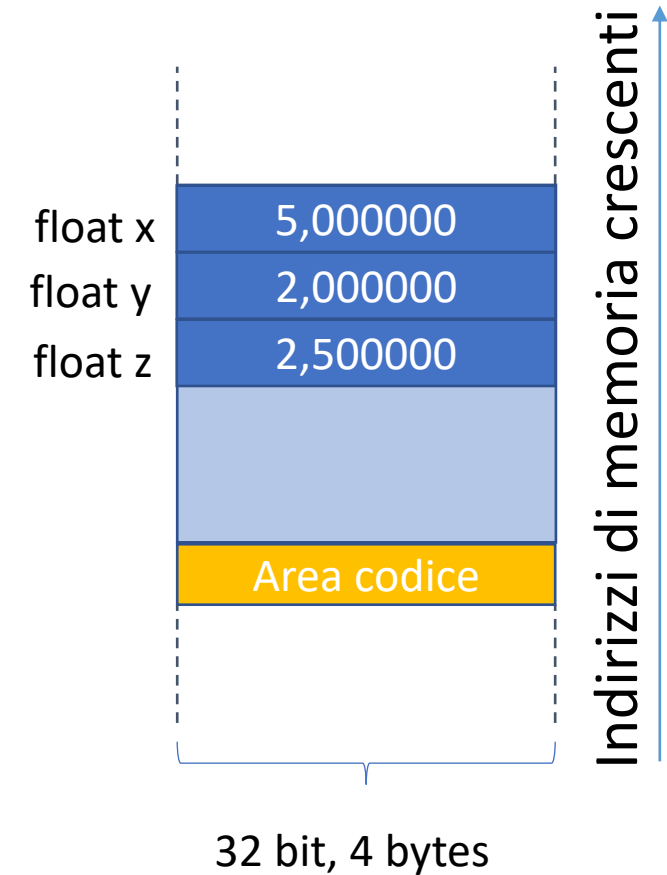
I tipi float e double

Tipo	Uso	CPU 32 bit		CPU 64 bit	
		Dimensione Tipica [Byte / bit]	Intervallo	Dimensione Tipica [Byte / bit]	Intervallo
char	ASCII	1 / 8	-128, 127	1 / 8	-128, 127
short	numeri interi complem. a due	2 / 16	$-2^{16}, 2^{16}-1$	2 / 16	$-2^{16}, 2^{16}-1$
int		4 / 32	$-2^{32}, 2^{32}-1$	4 / 32	$-2^{32}, 2^{32}-1$
long		4 / 32	$-2^{32}, 2^{32}-1$	8 / 64	$-2^{64}, 2^{64}-1$
long long		8 / 64	$-2^{64}, 2^{64}-1$	8 / 64	$-2^{64}, 2^{64}-1$
float	numeri con virgola	4 / 32	3.4E+/-38	4 / 32	3.4E+/-38
double		8 / 64	1,7E+/-308	8 / 64	1,7E+/-308

I tipi float e double

```
int main() {  
    float x = 5, y = 2, z;  
    z = x / y;  
    printf("La var x vale %f \n", x);  
    printf("La var y vale %f \n", y);  
    printf("La var z vale %f \n", z);  
}
```

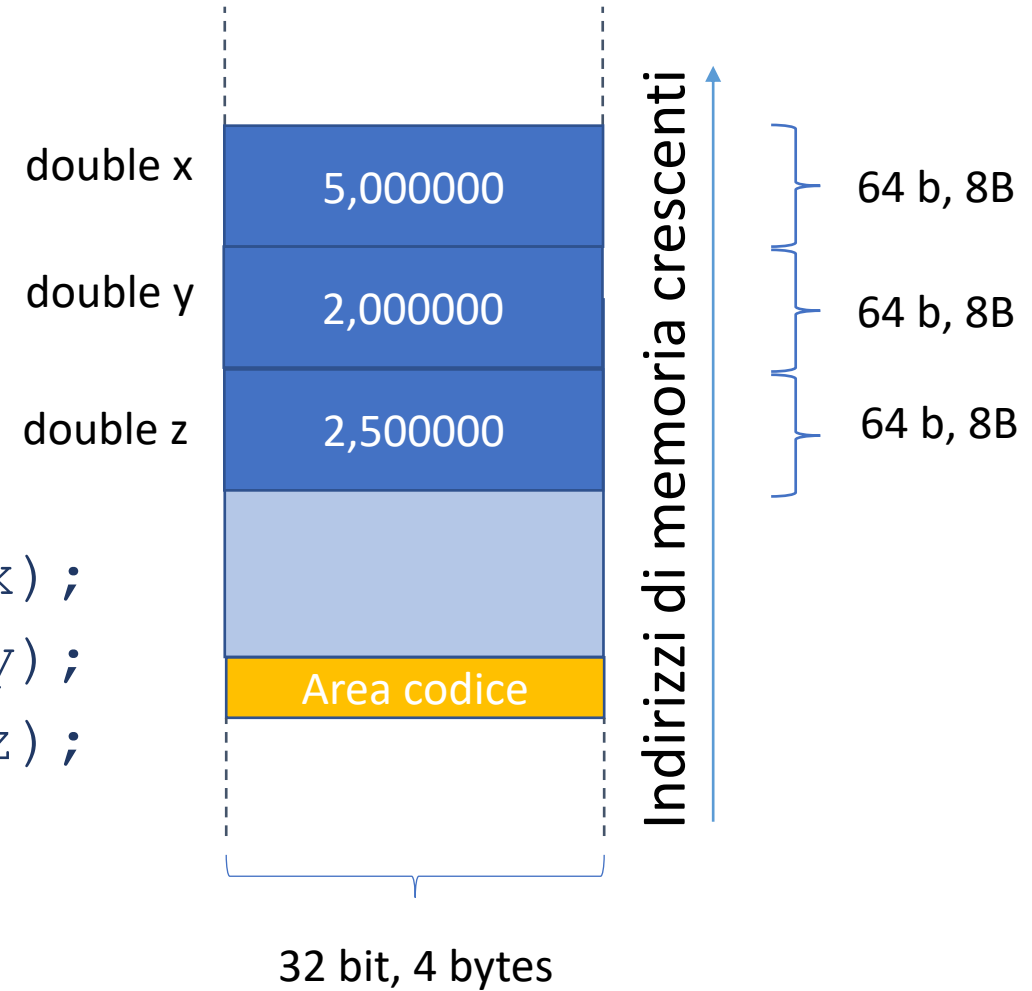
```
$ ./math  
La var x vale 5.000000  
La var y vale 2.000000  
La var z vale 2.500000
```



I tipi float e double

```
int main() {  
    double x = 5, y = 2, z;  
    z = x / y;  
    printf("La var x vale %f \n", x);  
    printf("La var y vale %f \n", y);  
    printf("La var z vale %f \n", z);  
}
```

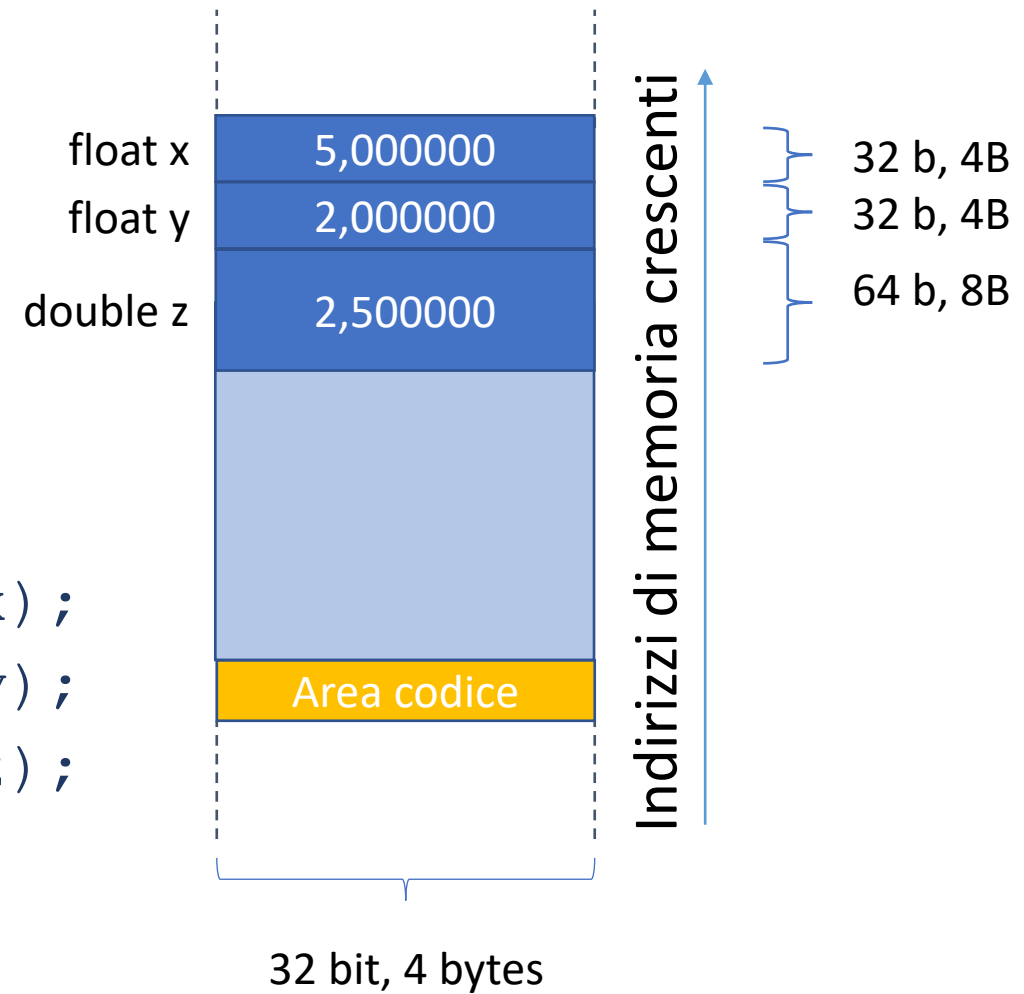
```
$ ./math  
La var x vale 5.000000  
La var y vale 2.000000  
La var z vale 2.500000
```



I tipi float e double

```
int main() {  
    float x = 5, y = 2;  
    double z;  
    z = x / y;  
    printf("La var x vale %f \n", x);  
    printf("La var y vale %f \n", y);  
    printf("La var z vale %f \n", z);  
}
```

```
$ ./math  
La var x vale 5.000000  
La var y vale 2.000000  
La var z vale 2.500000
```



La funzione printf

- Stampa notazione naturale

```
printf ("z vale %f\n", z);
```

z vale 2.500000

- Stampa troncata a terzo decimale

```
printf ("z vale %.3f\n", z);
```

z vale 2.500

- Stampa notazione esponenziale troncata terzo decimale

```
printf ("z vale %.3e\n", z);
```

z vale 2,500e+00

La libreria *math*

- Il linguaggio C supporta **nativamente** solo espressioni contenenti gli operatori corrispondenti alle 5 operazioni algebriche $*$ $/$ $\%$ $+$ $-$
- Per le altre funzioni matematiche (sin, cos, sqrt, pow) si può fare riferimento alle funzioni dichiarate nell'header file *math.h*
- Sono definite per tipi *int*, *float* e *double*

La libreria math

```
#include <stdio.h>
#include <math.h>
```

```
int main() {
    float num = 9.0, radice, quadrato;
    quadrato = pow(num, 2);
    radice = sqrt(num);
    printf ("Il quadrato di %.0f è %.2f\n", num
, quadrato);
    printf ("La radice di %.0f è %.2f", num ,
radice);
}
```

```
$ ./math_2
Il quadrato di 9 è 81.00
La radice di 9 è 3.00
```

Outline

- Numeri: float, aritmetica in virgola mobile, math.h
- Conversioni di tipo
- L'operatore di indirizzo
- Input da tastiera
- I puntatori

Conversioni di tipo implicite/esplicite

- È possibile scrivere espressioni con variabili di tipo diverso: per ora *int*, *float* e *double* (più avanti altri tipi_
- Il compilatore C effettua *implicitamente* o *esplicitamente* le opportune conversioni fra tipi: **casting**
- Per esempio l'assegnazione di un *float* a un *int* è effettuata implicitamente con eventuale perdita della parte frazionaria
- Nella conversione fra tipi è possibile una perdita di precisione senza che Il compilatore NON segnali errori o warning

Conversioni di tipo implicite

Quando si lavora con assegnazioni e tipi eterogenei è bene comprendere bene:

Conversioni di tipo implicite

Quando si lavora con assegnazioni e tipi eterogenei è bene comprendere bene:

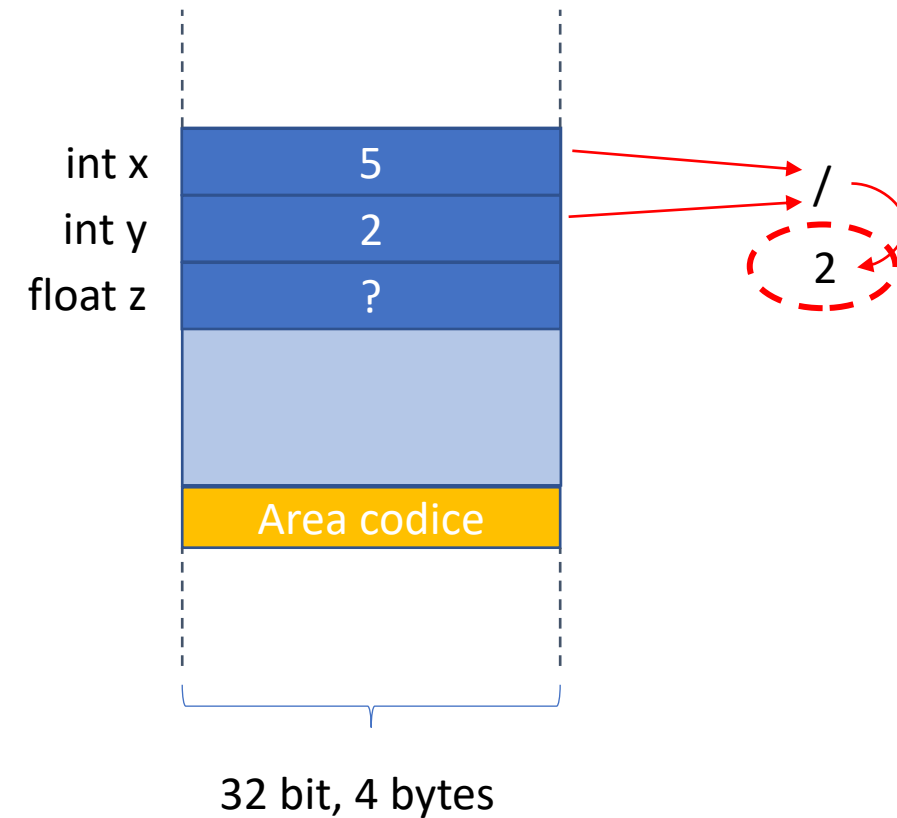
1. Il significato **dell'espressione** a destra: verificare se possa esserci perdita di precisione a causa della **scelta dei tipi**

Conversioni di tipo implicite

Perdita parte frazionaria
durante la divisione
fra interi

```
int main() {  
    int x = 5, y = 2;  
    → float z = x / y;  
    printf("La var x vale %d \n", x);  
    printf("La var y vale %d \n", y);  
    printf("La var z vale %f \n", z);  
}
```

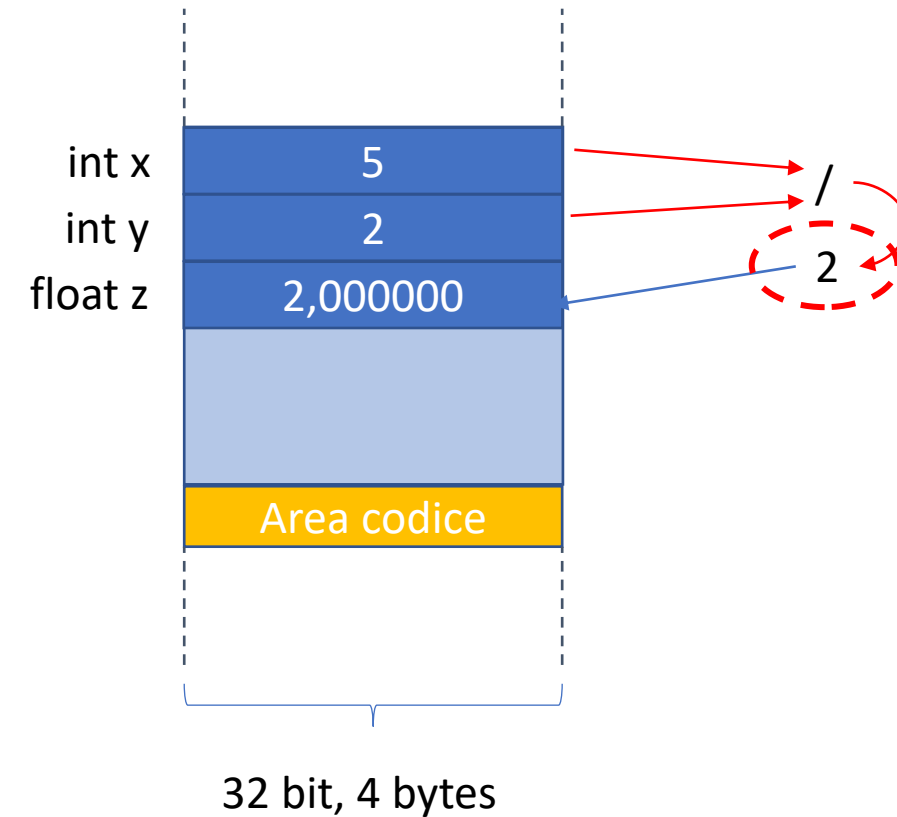
```
$ ./math
```



Conversioni di tipo implicite

```
int main() {  
    int x = 5, y = 2;  
    → float z = x / y;  
    printf("La var x vale %d \n", x);  
    printf("La var y vale %d \n", y);  
    printf("La var z vale %f \n", z);  
}
```

Perdita parte frazionaria
già avvenuta
all'assegnazione

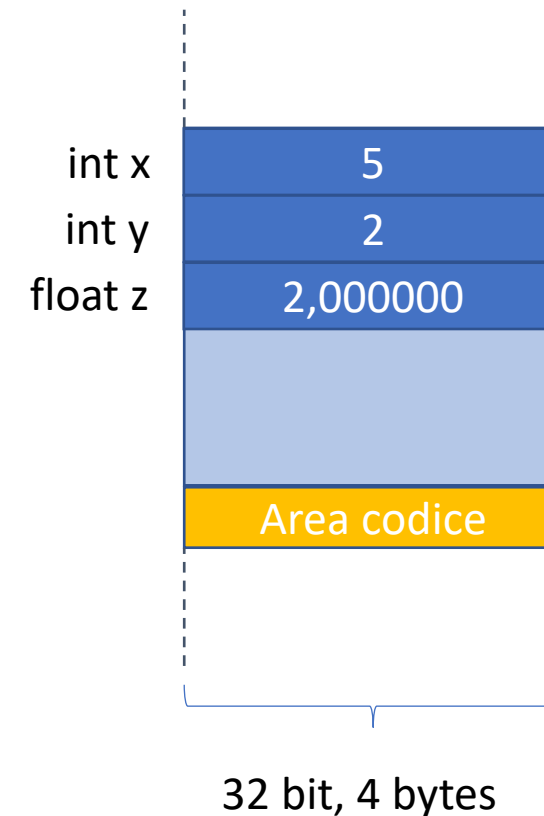


```
$ ./math
```

Conversioni di tipo implicite

```
int main() {  
    int x = 5, y = 2;  
    float z = x / y;  
→ printf("La var x vale %d \n", x);  
→ printf("La var y vale %d \n", y);  
→ printf("La var z vale %f \n", z);  
}
```

```
$ ./math  
La var x vale 5  
La var y vale 2  
La var z vale 2.000000
```



Conversioni di tipo implicite

Quando si lavora con assegnazioni e tipi eterogenei è bene comprendere bene:

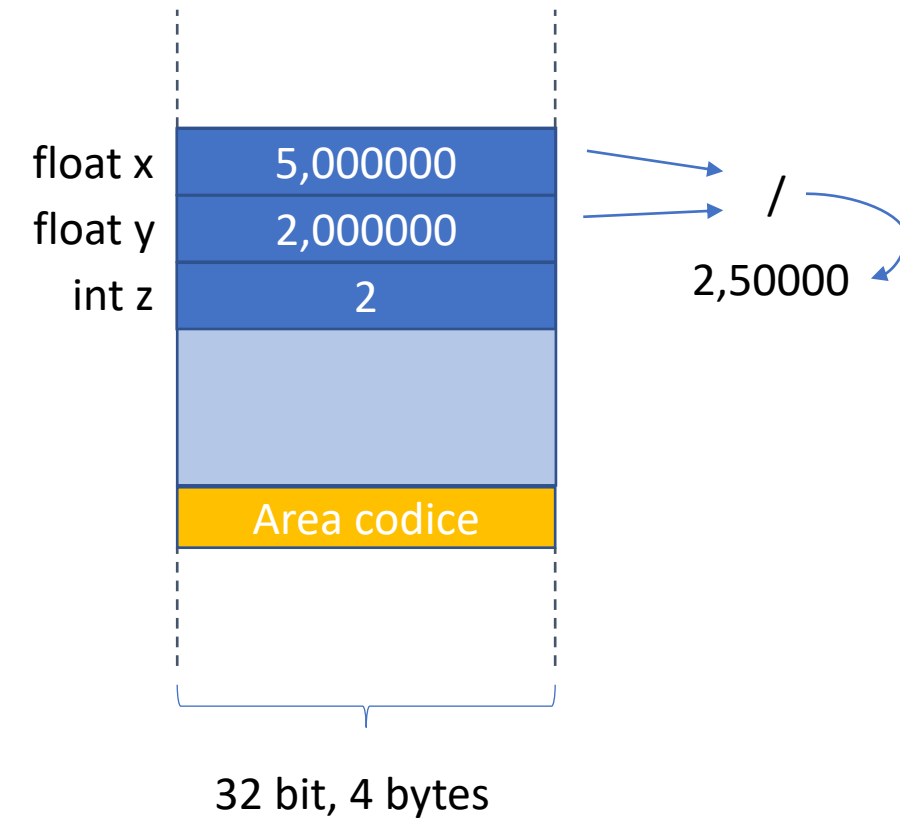
1. Il significato **dell'espressione** a destra: verificare se possa esserci perdita di precisione a causa della **scelta dei tipi**
2. Verificare che **l'operatore assegnazione** non sia la causa della perdita di precisione. L'assegnamento a *int* di valori *float* o *double* non produce warning al momento della compilazione

Conversioni di tipo implicite

```
int main() {  
    float x = 5, y = 2;  
    int z = x / y;  
    printf("La var x vale %f \n", x);  
    printf("La var y vale %f \n", y);  
    printf("La var z vale %d \n", z);  
}
```

Divisione fra float

```
$ ./math
```

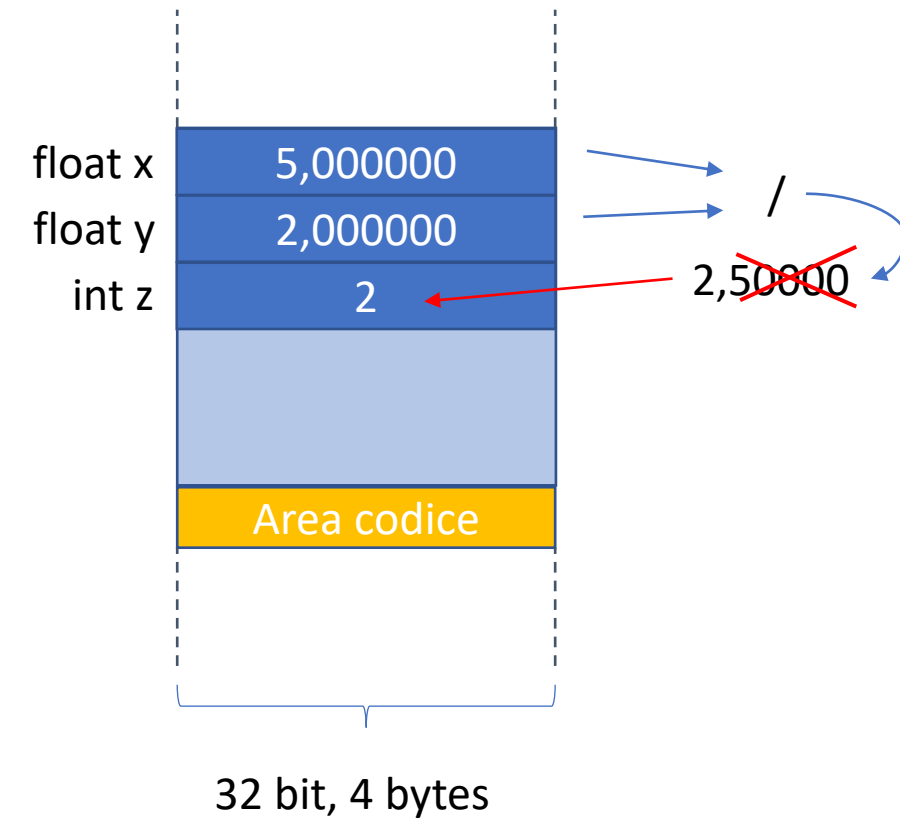


Conversioni di tipo implicite

Casting implicito float -> int
durante l'assegnazione
con perdita parte frazionaria

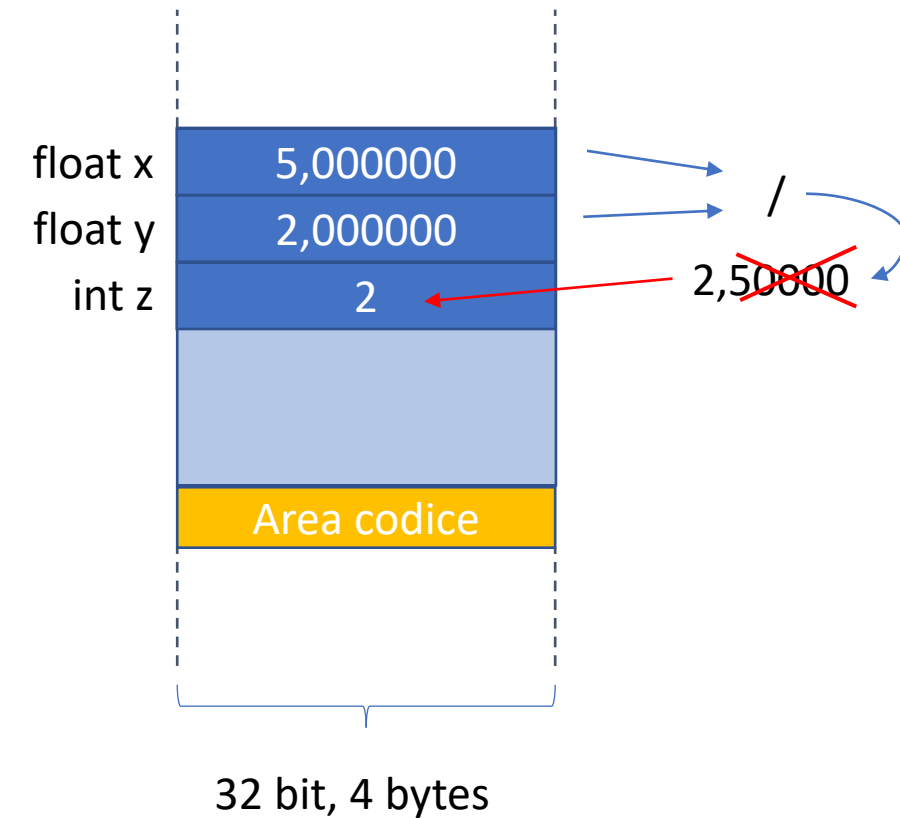
```
int main() {  
    float x = 5, y = 2;  
    int z = x / y;  
    printf("La var x vale %f \n", x);  
    printf("La var y vale %f \n", y);  
    printf("La var z vale %d \n", z);  
}
```

```
$ ./math
```



Conversioni di tipo implicite

```
int main() {  
    float x = 5, y = 2;  
    int z = x / y;  
    printf("La var x vale %f \n", x);  
    printf("La var y vale %f \n", y);  
    printf("La var z vale %d \n", z);  
}
```



```
$ ./math  
La var x vale 5.000000  
La var y vale 2.000000  
La var z vale 2
```

Parte frazionaria
persa

Conversioni di tipo esplicite

- È possibile forzare una rappresentazione di formato tramite conversione esplicita (**casting**)

```
int x = ...;  
float z = (float) x;
```

Operatore di casting a *float*

- Utile praticamente in situazioni come la seguente, notare la conversione del divisore

```
float z = x / (float) y;
```

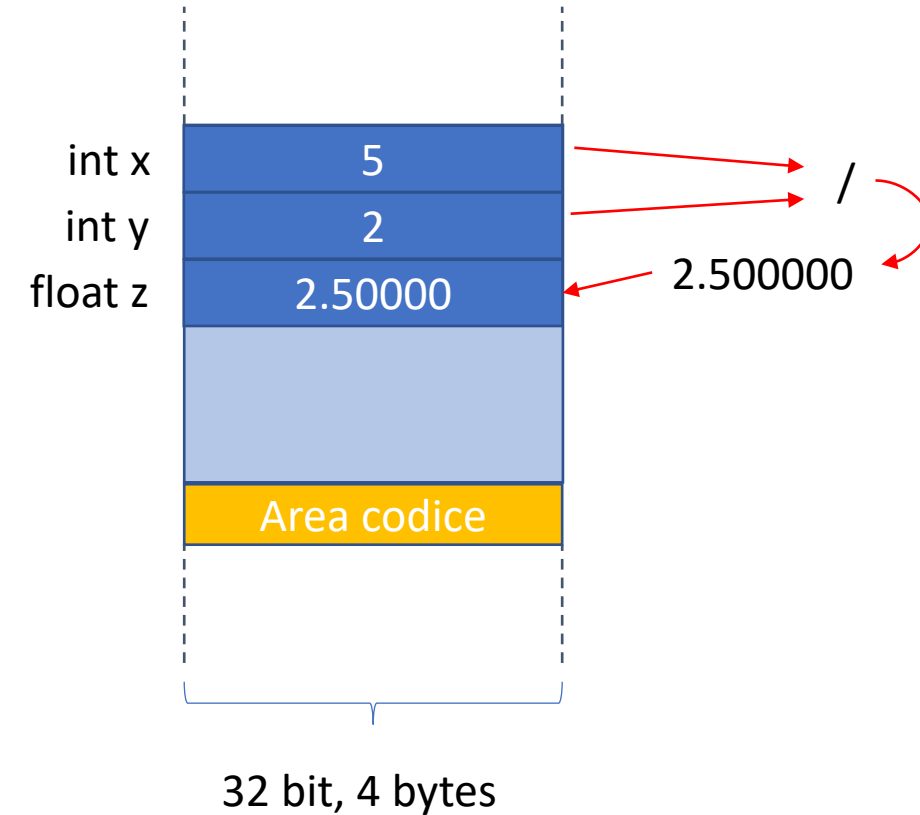
y convertita a *float*

Conversioni di tipo esplicite

```
int main() {  
    int x = 5, y = 2;  
    float z = x / (float)y;  
    printf("La var x vale %d \n", x);  
    printf("La var y vale %d \n", y);  
    printf("La var z vale %f \n", z);  
}
```

Casting di *y* a *float*
preserva la
parte frazionaria

```
$ ./math  
La var x vale 5  
La var y vale 2  
La var z vale 2.500000
```



Outline

- Numeri: float, aritmetica in virgola mobile, math.h
- Conversioni di tipo
- L'operatore di indirizzo
- Input da tastiera
- I puntatori

Indirizzo di una variabile

- Il linguaggio C permette *quasi sempre* di programmare ignorando l'effettivo indirizzo a runtime di una variabile
- Potremmo necessitare di conoscere l'indirizzo in memoria di una variabile al di là del suo valore
- L'indirizzo di una variabile è accessibile tramite **l'operatore di indirizzo &** (referenziamento, *referencing*, *ampersand*)
- L'espressione **&x** significa '*l'indirizzo di x*'

Indirizzo di una variabile

- Per il momento stamperemo a schermo gli indirizzi di variabili già dichiarate in precedenza (non necessariamente definiti)
- `printf` può stampare indirizzi utilizzando lo specificatore `%p`

```
int x;  
printf ("L'indirizzo di x è %p\n", &x);
```
- Gli indirizzi sono visualizzati in *notazione esadecimale* **0x.....** per compattezza, ma non sono altro che numeri interi senza segno

Indirizzo di una variabile

```
int main() {  
    int x = 5;  
    int y = 2;  
    int z = x + y;  
  
    printf("La var x all'indirizzo %p vale %d \n", &x, x);  
    printf("La var y all'indirizzo %p vale %d \n", &y, y);  
    printf("La var z all'indirizzo %p vale %d \n", &z, z);  
}
```

```
$ ./address  
La var x all'indirizzo 0x7fffd0624e0c vale 5  
La var y all'indirizzo 0x7fffd0624e08 vale 2  
La var z all'indirizzo 0x7fffd0624e04 vale 7
```

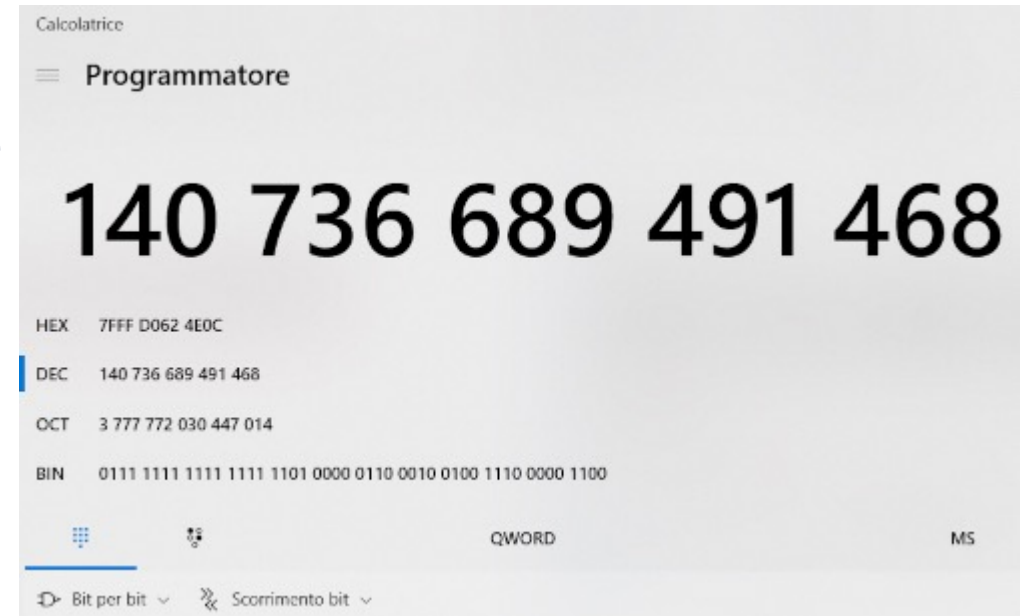
← I tre indirizzi sono in
notazione esadecimale

Indirizzo di una variabile

```
int main() {  
    int x = 5;  
    int y = 2;  
    int z = x + y;
```

```
    printf("La var x all'indirizzo %p vale %d \n", &x, x);  
    printf("La var y all'indirizzo %p vale %d \n", &y, y);  
    printf("La var z all'indirizzo %p vale %d \n", &z, z);
```

```
}
```



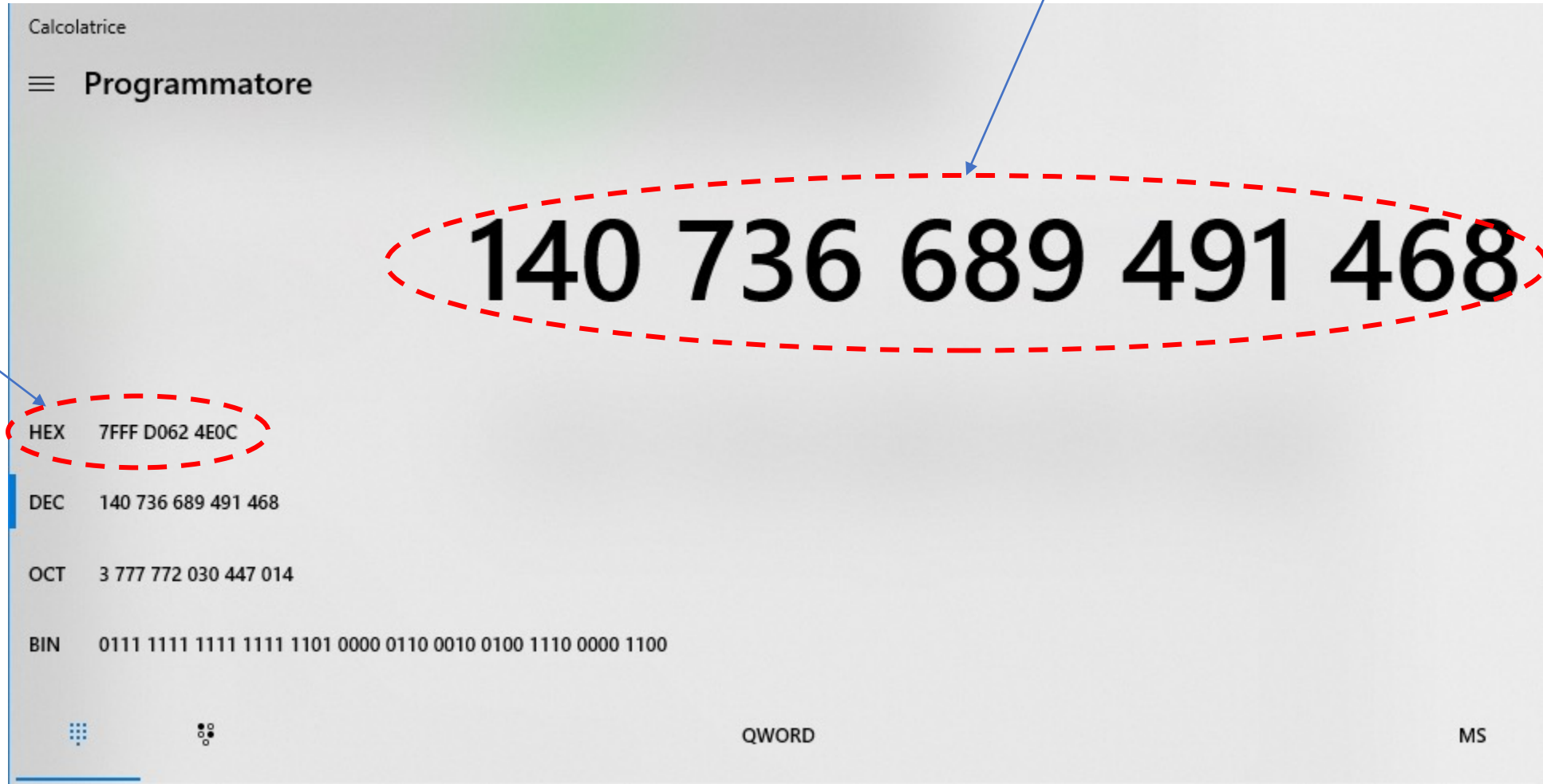
```
$ ./address  
La var x all'indirizzo 0x7fffd0624e0c vale 5  
La var y all'indirizzo 0x7fffd0624e08 vale 2  
La var z all'indirizzo 0x7fffd0624e04 vale 7
```

← I tre indirizzi sono in notazione esadecimale

Indirizzo di una variabile

Indirizzo in decimale
(circa $140 \cdot 10^{12} \sim \text{TB}$)

Indirizzo in
esadecimale



Indirizzo di una variabile

- Su una moderna macchina a 64 bit gli indirizzi di variabile tipicamente eccederanno il numero massimo di bytes gestibili dalla nostra ipotetica CPU a 32 bit (2^{32} , 4GB)
- Gli indirizzi di variabile tipicamente eccederanno la quantità di memoria fisica installata sulla macchina: si tratta infatti di indirizzi *virtuali* (argomento di Sistemi Operativi)

Indirizzo di una variabile

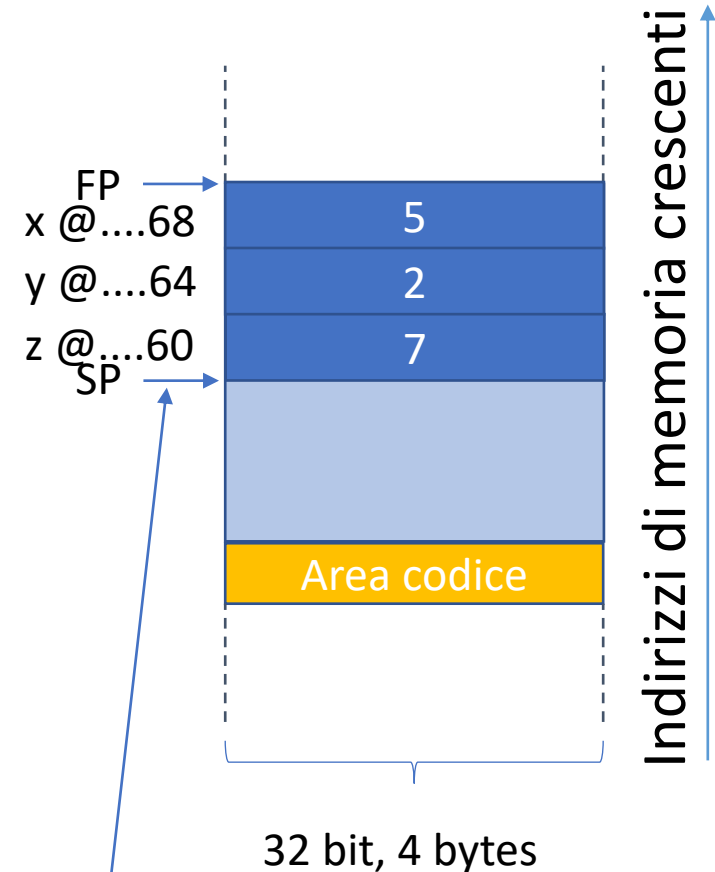
```
int main() {  
    int x = 5;  
    int y = 2;  
    int z = x + y;  
  
    printf("La var x all'indirizzo %p  
vale %d \n", &x, x);  
    printf("La var y all'indirizzo %p  
vale %d \n", &y, y);  
    printf("La var z all'indirizzo %p  
vale %d \n", &z, z);  
}
```

```
$ ./address
```

```
La var x all'indirizzo 140736689491468 vale 5
```

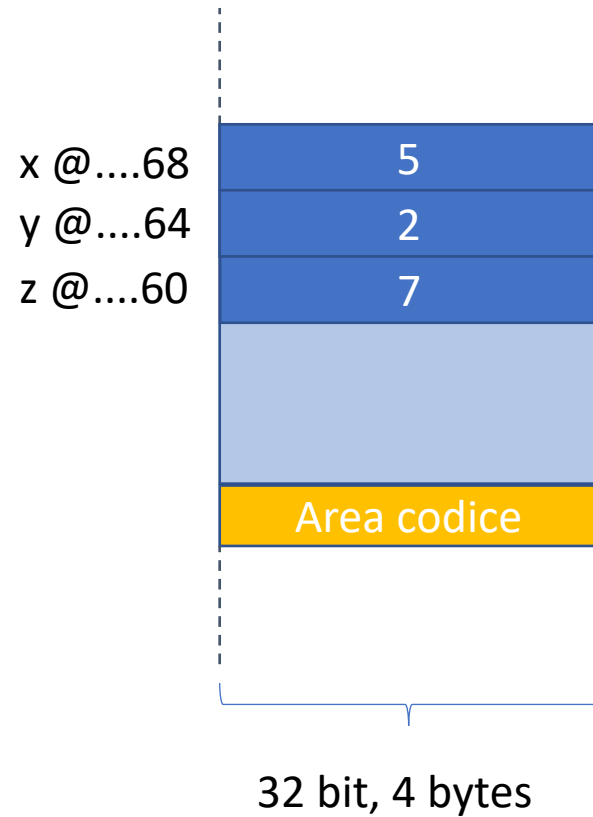
```
La var y all'indirizzo 140736689491464 vale 2
```

```
La var z all'indirizzo 140736689491460 vale 7
```

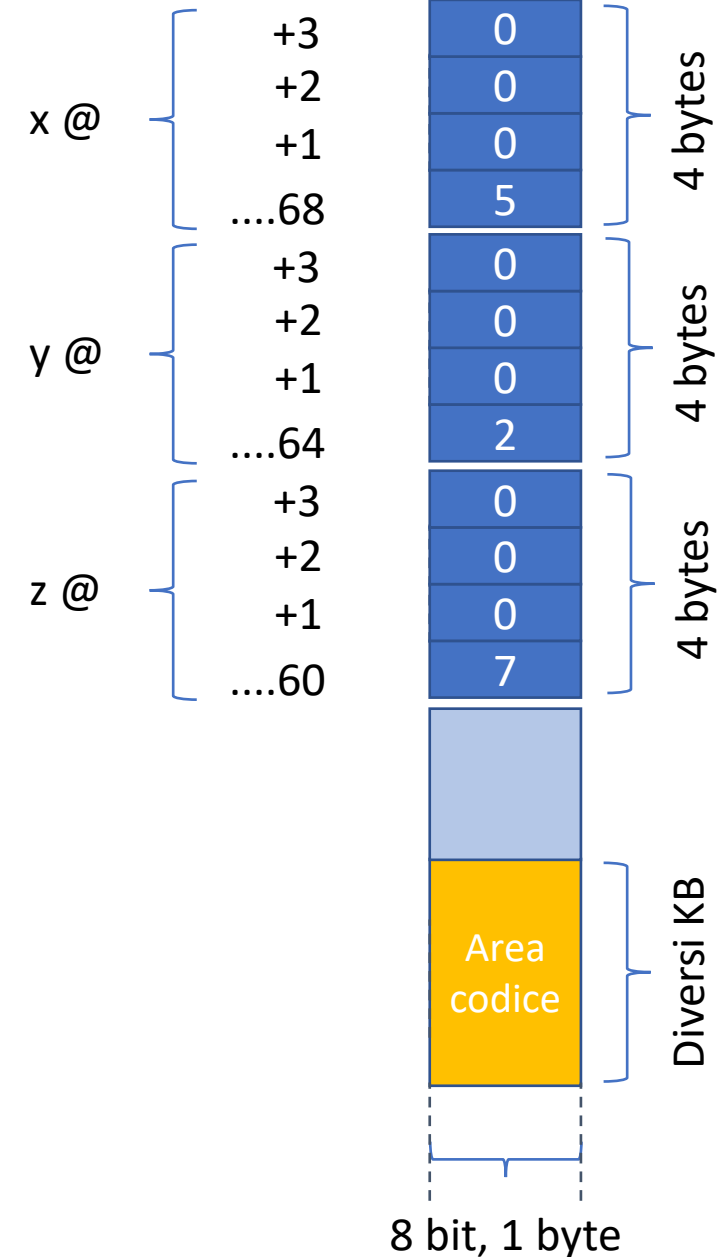


I tre indirizzi sono contigui e differiscono esattamente di 4 bytes a decrescere

Indirizzo di una variabile



Indirizzi di memoria crescenti



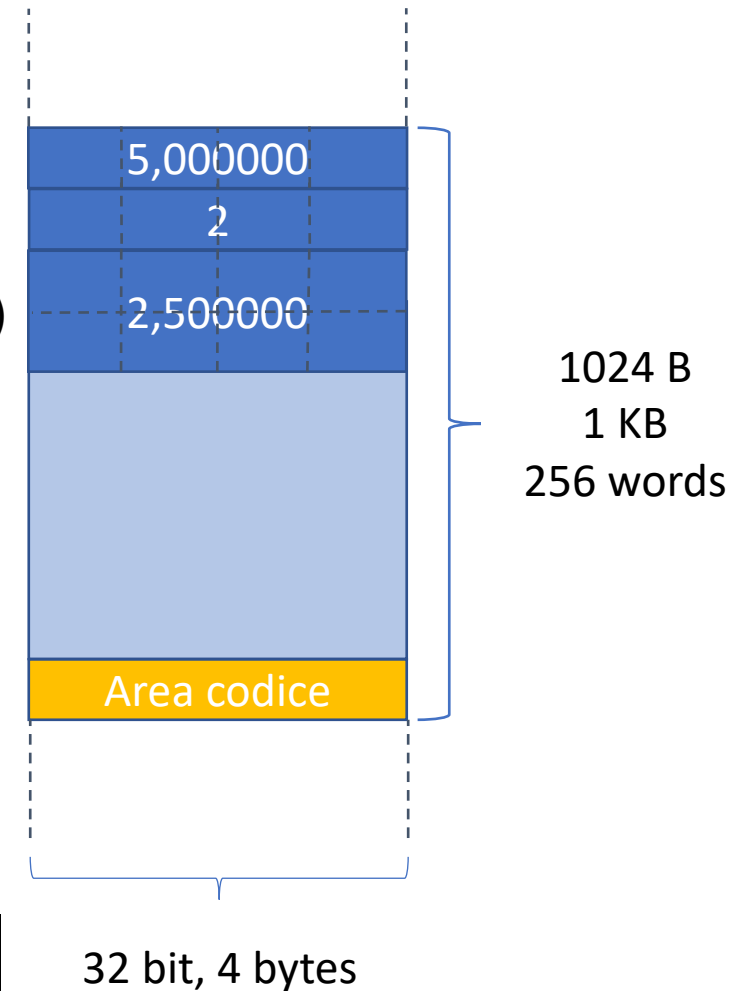
Indirizzo di una variabile

```
void main() {  
    float x = 5.0;  
    int y = 2;  
    double z = x/y;  
    printf("La var x all'ind %p vale %f  
    \n", &x, x);  
    printf("La var y all'ind %p vale %d  
    \n", &y, y);  
    printf("La var z all'ind %p vale %f  
    \n", &z, z);  
}
```

float x @0x03FC (1020₁₀)

int y @0x03F8 (1016₁₀)

double z @0x03F0 (1008₁₀)



```
$ ./math
```

```
La var x all'ind 0x000003FC vale 5.000000
```

```
La var y all'ind 0x000003FC vale 2
```

```
La var z all'ind 0x000003F0 vale 2.500000
```

Outline

- Numeri: float, aritmetica in virgola mobile, math.h
- Conversioni di tipo
- L'operatore di indirizzo
- Input da tastiera
- I puntatori

Input da tastiera

- La funzione **scanf** definita in `stdio.h` permette di leggere input formattato da tastiera
- **scanf** supporta input complessi, ma noi (per ora) la utilizzeremo per leggere esclusivamente singoli valori interi senza o con segno
- La sintassi per leggere un intero da tastiera e memorizzarlo in nella variabile intera già dichiarata `x` è `scanf ("%d", indirizzo_di_x)`

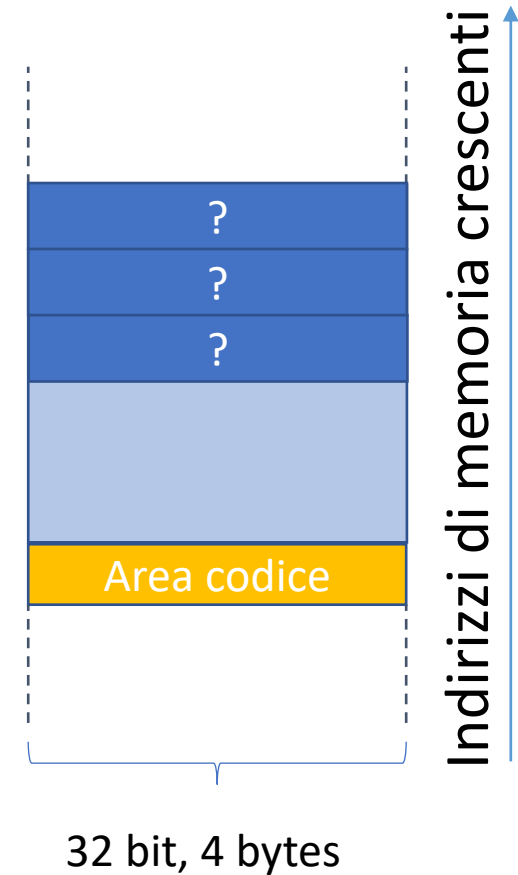
scanf (" %d" , &x)

ATTENZIONE: la lettura viene eseguito al momento della pressione del tasto ENTER

Input da tastiera

```
int main() {  
→ int x, y, z;  
→ printf("Inserisci x: ");  
  scanf("%d", &x);  
  printf("Inserisci y: ");  
  scanf("%d", &y);  
  z = x + y;  
  printf("%d + %d = %d\n", x, y, z);  
}
```

int x @0x03FC (1020₁₀)
int y @0x03F8 (1016₁₀)
int z @0x03F4 (1012₁₀)



```
$ ./scanf_ex1  
Inserisci x:
```

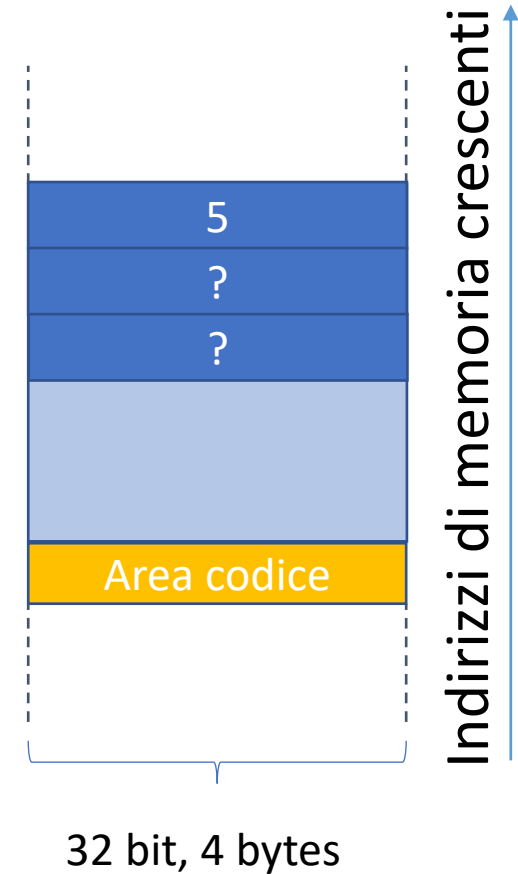
Input da tastiera

```
int main() {  
    int x, y, z;  
    printf("Inserisci x: ");  
→ scanf("%d", &x);  
    printf("Inserisci y: ");  
    scanf("%d", &y);  
    z = x + y;  
    printf("%d + %d = %d\n", x, y, z);  
}
```

int x @0x03FC (1020₁₀)

int y @0x03F8 (1016₁₀)

int z @0x03F4 (1012₁₀)



```
$ ./scanf_ex1  
Inserisci x: 5
```

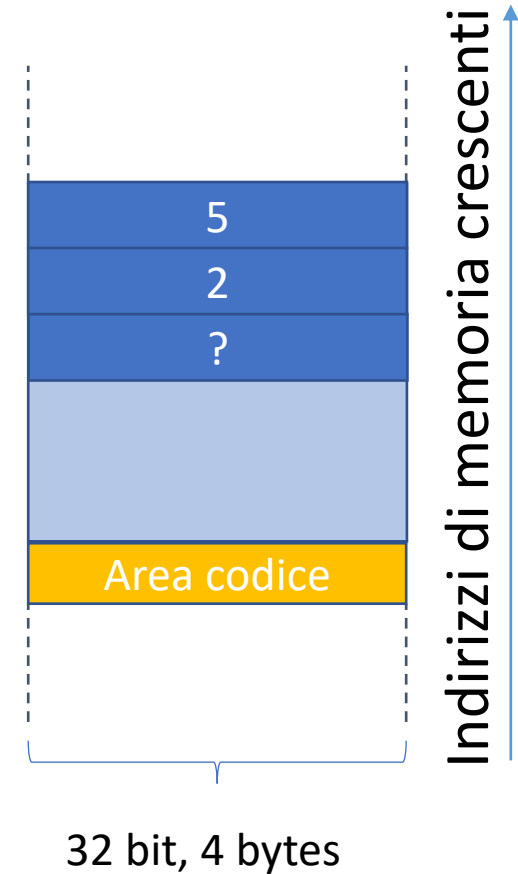
Input da tastiera

```
int main() {  
    int x, y, z;  
    printf("Inserisci x: ");  
    scanf("%d", &x);  
→ printf("Inserisci y: ");  
→ scanf("%d", &y);  
    z = x + y;  
    printf("%d + %d = %d\n", x, y, z);  
}
```

int x @0x03FC (1020₁₀)

int y @0x03F8 (1016₁₀)

int z @0x03F4 (1012₁₀)

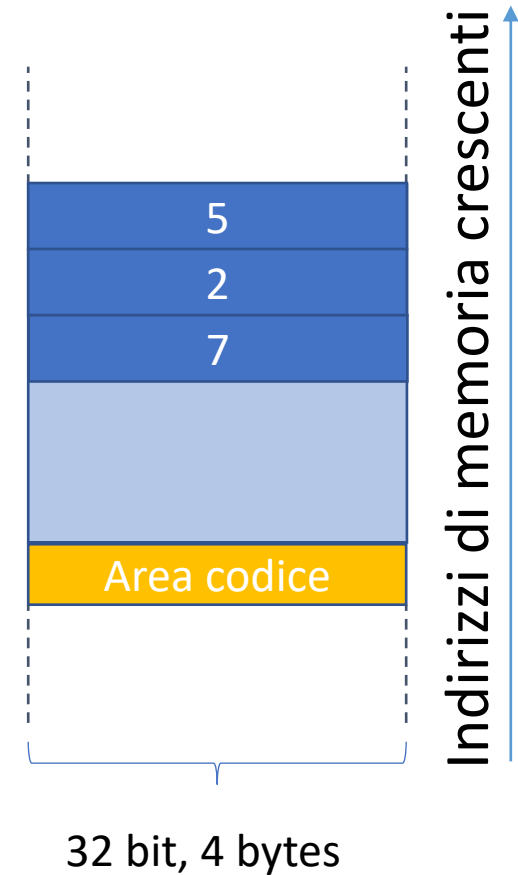


```
$ ./scanf_ex1  
Inserisci x: 5
```

Input da tastiera

```
int main() {  
    int x, y, z;  
    printf("Inserisci x: ");  
    scanf("%d", &x);  
    printf("Inserisci y: ");  
    scanf("%d", &y);  
→ z = x + y;  
→ printf("%d + %d = %d\n", x, y, z);  
}
```

int x @0x03FC (1020₁₀)
int y @0x03F8 (1016₁₀)
int z @0x03F4 (1012₁₀)



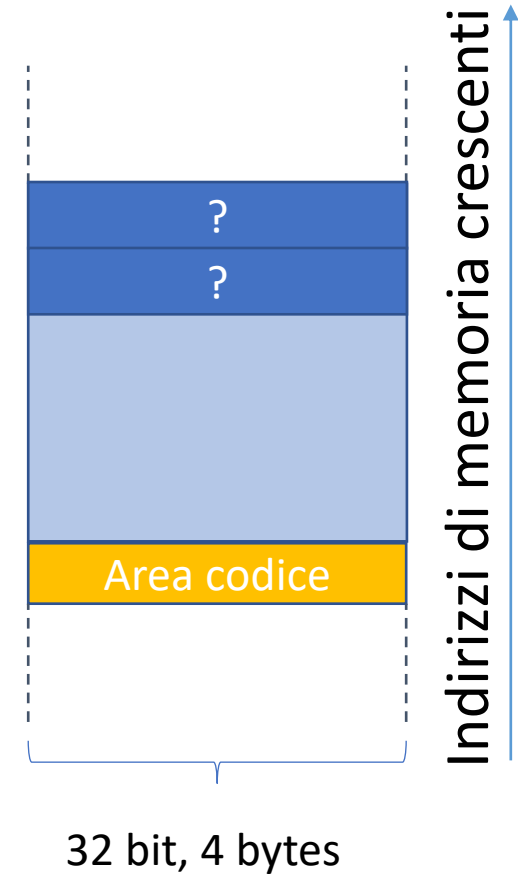
```
$ ./scanf_ex1  
Inserisci x: 5  
Inserisci y: 2  
5 + 2 = 7
```


Input da tastiera

```
int main() {  
→ int x, y, z;  
→ printf("Inserisci due interi: ");  
→ scanf("%d %d", &x, &y);  
  z = x + y;  
  printf("%d + %d = %d\n", x, y, z);  
}
```

carattere
di spazio

int x @0x03FC (1020₁₀)
int y @0x03F8 (1016₁₀)



```
$ ./scanf_ex2  
Inserisci due interi:
```

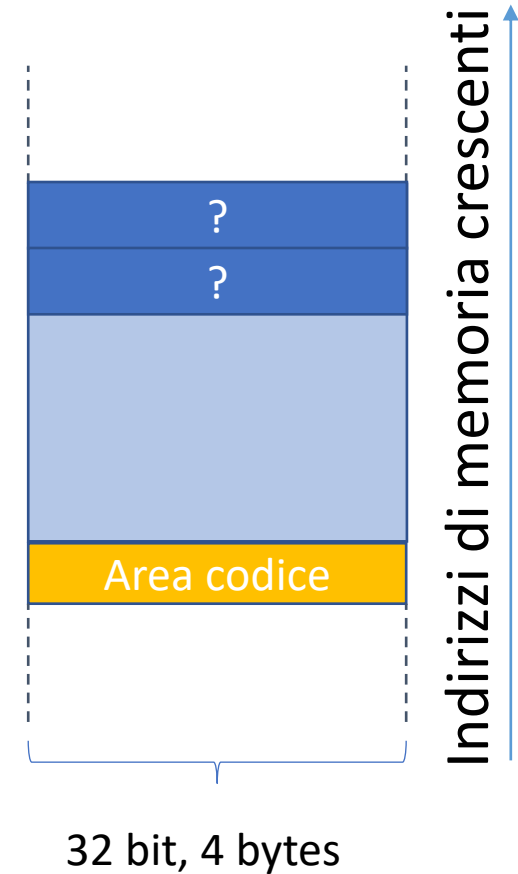
Input da tastiera

```
int main() {  
→ int x, y, z;  
→ printf("Inserisci due interi: ");  
→ scanf("%d %d", &x, &y);  
  z = x + y;  
  printf("%d + %d = %d\n", x, y, z);  
}
```

carattere
di spazio

```
$ ./scanf_ex2  
Inserisci due interi: 5 2
```

int x @0x03FC (1020₁₀)
int y @0x03F8 (1016₁₀)

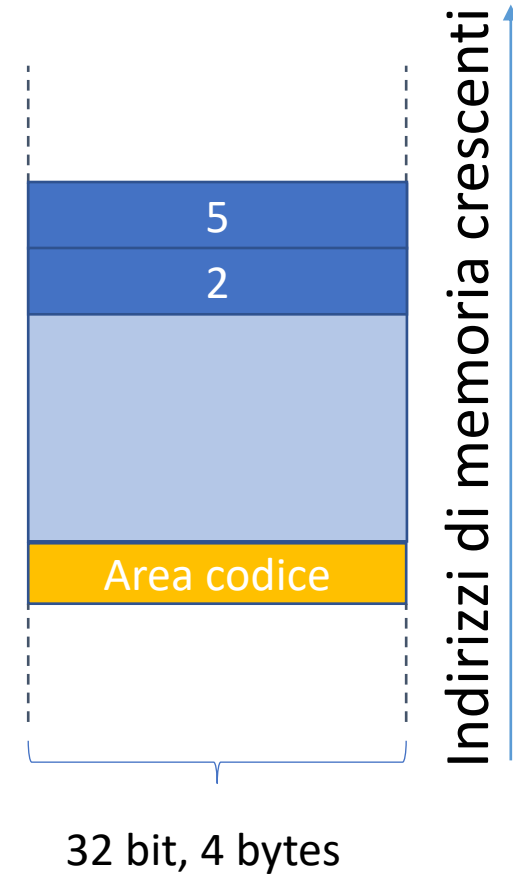


Input da tastiera

```
int main() {  
    int x, y, z;  
    printf("Inserisci due interi : ");  
→ scanf("%d %d", &x, &y);  
    z = x + y;  
    printf("%d + %d = %d\n", x, y, z);  
}
```

int x @0x03FC (1020₁₀)

int y @0x03F8 (1016₁₀)



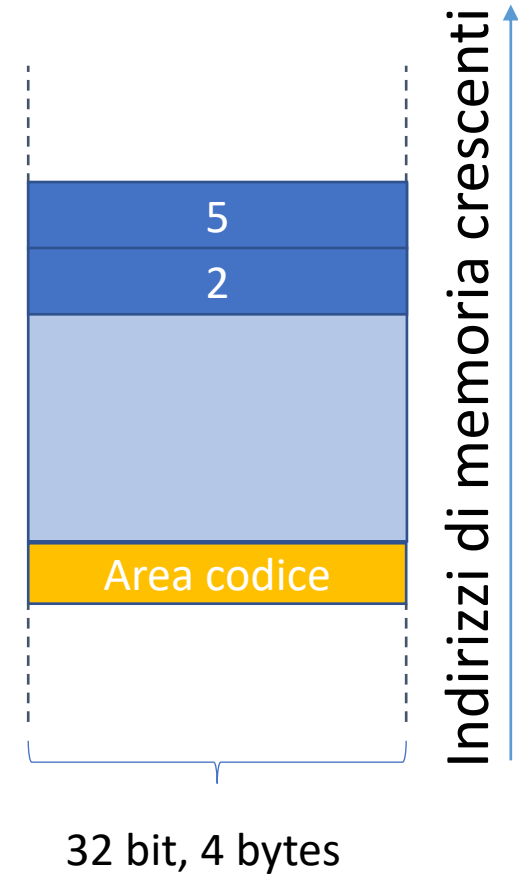
```
$ ./scanf_ex2  
Inserisci due interi: 5 2
```

Input da tastiera

```
int main() {  
    int x, y, z;  
    printf("Inserisci due interi : ");  
    scanf("%d %d", &x, &y);  
→ z = x + y;  
→ printf("%d + %d = %d\n", x, y, z);  
}
```

int x @0x03FC (1020₁₀)

int y @0x03F8 (1016₁₀)



```
$ ./scanf_ex2  
Inserisci due interi: 5 2  
5 + 2 = 7
```

Input da tastiera

- La funzione `scanf` analizza l'input utente ma non si blocca nel caso di errori (es., *elemento mancante o tipo errato*)
- La funzione ritorna però il **numero** di elementi correttamente inseriti dall'utente secondo quanto specificato

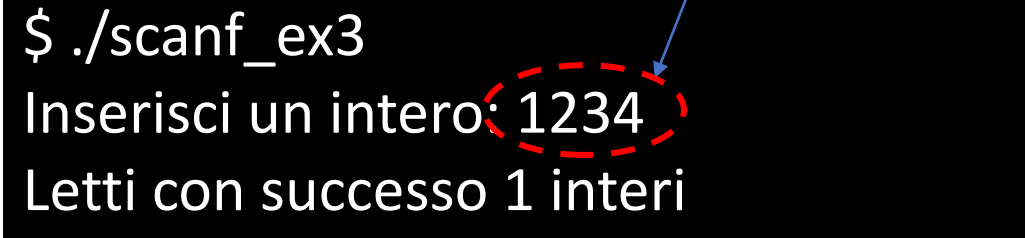
```
int x, y;  
int n = scanf ("%d %d", &x, &y)
```



n vale due se e solo se l'utente inserisce
due numeri interi separati da uno spazio

Input da tastiera

```
int main() {  
    int x;  
    printf("Inserisci un intero: ");  
    int n = scanf("%d", &x);  
    printf("Letti con successo %d interi\n", n);  
}
```

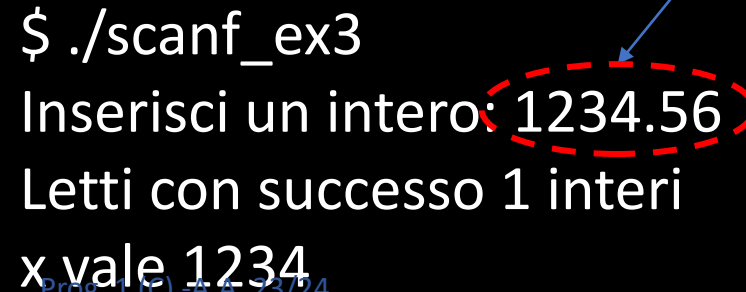


\$./scanf_ex3
Inserisci un intero: 1234
Letti con successo 1 interi

Input da tastiera

```
int main() {  
    int x;  
    printf("Inserisci un intero: ");  
    int n = scanf("%d", &x);  
    printf("Letti con successo %d interi\n", n);  
    printf("x vale %d \n", x);  
}
```

Numero
con virgola



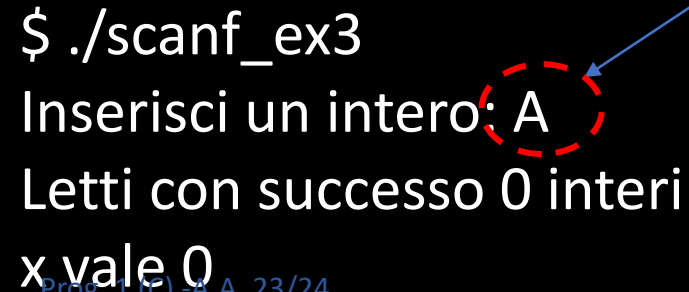
```
$ ./scanf_ex3  
Inserisci un intero: 1234.56  
Letti con successo 1 interi  
x vale 1234
```

Input da tastiera

```
int main() {  
    int x = 0;  
    printf("Inserisci un intero: ");  
    int n = scanf("%d", &x);  
    printf("Letti con successo %d interi\n", n);  
    printf("x vale %d \n", x);  
}
```

Variabile x
non aggiornata

Carattere
anzichè intero



```
$ ./scanf_ex3  
Inserisci un intero: A  
Letti con successo 0 interi  
x vale 0
```